

Four scripts that run, and four that do not learn

Every script below runs without an error and every one of them is wrong. One bug each — a learning rate, a scale, a dead layer, a label mix-up. Read the evidence, find the line, fix it.

Scaffolded lab

22 minutes

Debugging

One bug per script

Before you start

Setup · given

1 · The loss that jumps

2 · The loss that stalls...

3 · The hidden layer that...

4 · The model that learns...

Reflection

If you finish early

Before you start

Open the provided `lab13.ipynb`. The training function and a `report` helper are given; each script is a few lines that call them, with one line wrong.



By the end of this lab you can

- Recognise four failure signatures in a loss curve.
- Trace each signature back to one line of code.
- Fix it, and prove the fix with a number rather than a feeling.

LAB 13

Setup · given

Lab 11's data and a compact version of Lab 11's loop. A correct run reaches about **96% test accuracy** in 30 epochs — keep that number in mind as the healthy reference.

```

import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits

digits = load_digits()
X = digits.data / 16.0 # 1797 images, 64 pixels each,
# scaled to 0-1
y = (digits.target % 2 == 0).astype(float).reshape(-1, 1) # 1 = even digit, 0 = odd digit

order = np.random.default_rng(0).permutation(len(X))
X_train, y_train = X[order[:1347]], y[order[:1347]]
X_test, y_test = X[order[1347:]], y[order[1347:]]
print("train", X_train.shape, " test", X_test.shape, " share of even digits:",
      round(y_train.mean(), 3))

def sigmoid(z):
    return 1 / (1 + np.exp(-np.clip(z, -500, 500)))

def train_mlp(X, y, hidden=16, activation="sigmoid", lr=1.0, epochs=30, b1_init=0.0, seed=7):
    """Lab 11's loop, with a choice of hidden activation. Returns the loss curve and a
    predict function."""
    rng = np.random.default_rng(seed)
    p = {"W1": rng.normal(0, 0.1, (X.shape[1], hidden)), "b1": np.full((1, hidden),
float(b1_init)),
        "W2": rng.normal(0, 0.1, (hidden, 1)), "b2": np.zeros((1, 1))}
    act = (lambda z: np.maximum(z, 0)) if activation == "relu" else sigmoid
    slope = (lambda Z, A: (Z > 0).astype(float)) if activation == "relu" else (lambda Z, A: A
* (1 - A))
    order_rng, curve = np.random.default_rng(1), []
    with np.errstate(all="ignore"):
        for _ in range(epochs):
            idx = order_rng.permutation(len(X))
            for s in range(0, len(X), 32):
                b = idx[s:s + 32]
                Z1 = X[b] @ p["W1"] + p["b1"]; A1 = act(Z1); A2 = sigmoid(A1 @ p["W2"] +
p["b2"])

                dZ2 = (A2 - y[b]) / len(b); dZ1 = (dZ2 @ p["W2"].T) * slope(Z1, A1)
                g = {"W2": A1.T @ dZ2, "b2": dZ2.sum(0, keepdims=True), "W1": X[b].T @ dZ1,
"b1": dZ1.sum(0, keepdims=True)}
                for k in p: p[k] -= lr * g[k]
                A2 = sigmoid(act(X @ p["W1"] + p["b1"]) @ p["W2"] + p["b2"])
                curve.append(float(-np.mean(y * np.log(A2 + 1e-12) + (1 - y) * np.log(1 - A2 +
1e-12))))
            predict = lambda Xn: sigmoid(act(Xn @ p["W1"] + p["b1"]) @ p["W2"] + p["b2"]) > 0.5
            hidden_on = lambda Xn: float(np.mean(Xn @ p["W1"] + p["b1"] > 0))
            return curve, predict, hidden_on

def report(name, curve, predict, Xte=None):
    Xte = X_test if Xte is None else Xte
    acc = float(np.mean(predict(Xte) == y_test))
    plt.figure(figsize=(5, 2.6)); plt.plot(curve); plt.xlabel("epoch"); plt.ylabel("training
loss"); plt.title(name); plt.show()

```

```
print(f"{name}: first-epoch loss {curve[0]:.3f}, last {curve[-1]:.3f}, test accuracy {acc:.3f}")
return acc

curve, predict, _ = train_mlp(X_train, y_train)
healthy = report("healthy reference", curve, predict)
```

what it should print

OUTPUT

```
train (1347, 64) test (450, 64) share of even digits: 0.483
healthy reference: first-epoch loss 0.859, last 0.056, test accuracy 0.962
```

SCRIPT 1

The loss that jumps

1

Run it as it is. Then, in the cell underneath: **(a)** name the symptom in the curve, **(b)** find the one line responsible, **(c)** fix it and rerun until the assert passes.

Look at: whether the loss goes down at all, and how much it moves from one epoch to the next.

script 1 – broken on purpose

PYTHON

```
curve, predict, _ = train_mlp(X_train, y_train, lr=50.0)
report("script 1", curve, predict)
```

what it should print

OUTPUT

```
script 1: first-epoch loss 14.009, last 7.757, test accuracy 0.533
```

your turn

PYTHON

```
# TODO: your fixed copy of script 1. Change exactly one line.
curve, predict, _ = train_mlp(X_train, y_train, lr=50.0)
acc = report("script 1 fixed", curve, predict)
assert acc > 0.9
```

one version that works

PYTHON

```
curve, predict, _ = train_mlp(X_train, y_train, lr=1.0)
acc = report("script 1 fixed", curve, predict)
assert acc > 0.9
```

what it should print

OUTPUT

```
script 1 fixed: first-epoch loss 0.859, last 0.056, test accuracy 0.962
```

Script 1 – (a) symptom:

(b) the line:

(c) why the fix works:

SCRIPT 2

The loss that stalls and stutters

2

Run it as it is. Then, in the cell underneath: **(a)** name the symptom in the curve, **(b)** find the one line responsible, **(c)** fix it and rerun until the assert passes.

Look at: the same learning rate as the healthy run, but inputs sixteen times larger. Lab 7's extension A met this already.

script 2 – broken on purpose

PYTHON

```
X_raw_train = digits.data[order[:1347]]          # pixel values 0-16
X_raw_test  = digits.data[order[1347:]]
curve, predict, _ = train_mlp(X_raw_train, y_train)
report("script 2", curve, predict, X_raw_test)
```

what it should print

OUTPUT

```
script 2: first-epoch loss 0.808, last 0.425, test accuracy 0.816
```

your turn

PYTHON

```
# TODO: your fixed copy of script 2. Change exactly one line.
X_raw_train = digits.data[order[:1347]]
X_raw_test  = digits.data[order[1347:]]
curve, predict, _ = train_mlp(X_raw_train, y_train)
acc = report("script 2 fixed", curve, predict, X_raw_test)
assert acc > 0.9
```

one version that works

PYTHON

```
X_raw_train = digits.data[order[:1347]] / 16.0
X_raw_test  = digits.data[order[1347:]] / 16.0
curve, predict, _ = train_mlp(X_raw_train, y_train)
acc = report("script 2 fixed", curve, predict, X_raw_test)
assert acc > 0.9
```

what it should print

OUTPUT

```
script 2 fixed: first-epoch loss 0.859, last 0.056, test accuracy 0.962
```

Script 2 — (a) symptom:

(b) the line:

(c) why the fix works:

SCRIPT 3

The hidden layer that went silent

3

Run it as it is. Then, in the cell underneath: **(a)** name the symptom in the curve, **(b)** find the one line responsible, **(c)** fix it and rerun until the assert passes.

Look at: the loss value it sits at — you have met that exact number twice before — and the last line printed.

script 3 – broken on purpose

PYTHON

```
curve, predict, hidden_on = train_mlp(X_train, y_train, activation="relu", lr=0.5,
b1_init=-3.0)
report("script 3", curve, predict)
print("share of hidden activations above zero:", hidden_on(X_train))
```

what it should print

OUTPUT

```
script 3: first-epoch loss 0.693, last 0.693, test accuracy 0.467
share of hidden activations above zero: 0.0
```

your turn

PYTHON

```
# TODO: your fixed copy of script 3. Change exactly one line.
curve, predict, hidden_on = train_mlp(X_train, y_train, activation="relu", lr=0.5,
b1_init=-3.0)
acc = report("script 3 fixed", curve, predict)
print("share of hidden activations above zero:", hidden_on(X_train))
assert acc > 0.9
```

one version that works

PYTHON

```
curve, predict, hidden_on = train_mlp(X_train, y_train, activation="relu", lr=0.5,
b1_init=0.0)
acc = report("script 3 fixed", curve, predict)
print("share of hidden activations above zero:", hidden_on(X_train))
assert acc > 0.9
```

what it should print

OUTPUT

```
script 3 fixed: first-epoch loss 1.058, last 0.015, test accuracy 0.973
share of hidden activations above zero: 0.3569506310319228
```

Script 3 – (a) symptom:

(b) the line:

(c) why the fix works:

SCRIPT 4

The model that learns nothing useful

4

Run it as it is. Then, in the cell underneath: **(a)** name the symptom in the curve, **(b)** find the one line responsible, **(c)** fix it and rerun until the assert passes.

Look at: the test accuracy next to the baseline of guessing. Nothing crashed — so what does the network think the labels are?

script 4 – broken on purpose

PYTHON

```
shuffle = np.random.default_rng(3).permutation(len(X_train))
X_mixed = X_train[shuffle]                    # "shuffle the training set before
training"
curve, predict, _ = train_mlp(X_mixed, y_train)
report("script 4", curve, predict)
```

what it should print

OUTPUT

```
script 4: first-epoch loss 0.736, last 0.702, test accuracy 0.538
```

your turn

PYTHON

```
# TODO: your fixed copy of script 4. Change exactly one line.
shuffle = np.random.default_rng(3).permutation(len(X_train))
X_mixed = X_train[shuffle]
curve, predict, _ = train_mlp(X_mixed, y_train)
acc = report("script 4 fixed", curve, predict)
assert acc > 0.9
```

one version that works

PYTHON

```
shuffle = np.random.default_rng(3).permutation(len(X_train))
X_mixed, y_mixed = X_train[shuffle], y_train[shuffle]
curve, predict, _ = train_mlp(X_mixed, y_mixed)
acc = report("script 4 fixed", curve, predict)
assert acc > 0.9
```

what it should print

OUTPUT

```
script 4 fixed: first-epoch loss 0.764, last 0.058, test accuracy 0.969
```

Script 4 — (a) symptom:

(b) the line:

(c) why the fix works:

Reflection



Write it in the notebook, in your own words

For each script, write the **first thing you would print** if you met that curve in your own project next month — before reading any code.

Your answer:

If you finish early

A. Script 3 used ReLU. Rerun it with `activation="sigmoid"` and the same `b1_init=-3.0`. Does the sigmoid layer die too? Explain the difference in one sentence using each function's slope.

B. Invent a fifth bug that runs silently, and give it to your neighbour without telling them where it is.

your turn

PYTHON

```
# TODO (optional extensions A, B)
```

